2023 IEEE 29th International Symposium on Local and Metropolitan Area Networks (LANMAN) | 979-8-3503-4693-0/23/$31.00 ©2023 IEEE | DOI: 10.1109/LANMAN58293.2023.10189416

Enhancing Reliability of Scheduled Traffic in Time-Sensitive Networks using Frame Replication and Elimination *

Soumya Kanta Rana, Himanshu Verma, Joydeep Pal,
Deepak Choudhary, T. V. Prabhakar, Chandramani Singh
Department of Electronic Systems Engineering
Indian Institute of Science, Bengaluru, India
e-mail: {soumyarana, himanshuv, joydeeppal, deepakcl, tvprabs, chandra}@iisc.ac.in

***Abstract*—The IEEE 802.1CB standard for Frame Replication and Elimination for Reliability (FRER) emphasises improving reliability by introducing link redundancy. In this paper, two approaches towards the elimination of duplicates are implemented on a programmable pipelined switch. The first approach meets the intermittent stream traffic goal of the standard. A sliding window-based algorithm is used for storing and comparing identification numbers of packets. The second approach handles bulk stream traffic by using a hash table along with the sliding window for a faster lookup, making it more scalable. Hash-collision mitigation with the help of stream identification is also incorporated in the hash table approach. The algorithms are implemented in P4 and Micro-C. Latency versus window size obtained for both the approaches and the end-to-end packet loss reduction yielded by FRER in presence of lossy intermediate links are demonstrated.**

***Index Terms*—Time Sensitive Networking (TSN), Frame Replication and Elimination for Reliability (FRER), P4, SmartNIC.**

I. Introduction

Bounded latency and reliability are the two pillars of Tactile Internet. In applications such as remote surgery, patients' vital parameters, pose of surgical tools, haptic feedback and video are required to be streamed in real-time across a network of Time Sensitive Network (TSN) switches. With the advent of 5G and the development of ultra-Reliable Low-Latency Communication (uRLLC) networks, the high data rates supported by the infrastructure can be used in several critical applications such as remote diagnostics and remote surgery [1].

Traffic through a TSN network can be broadly categorized into two classes: best-effort traffic (BE) and scheduled traffic (ST). The IEEE 802.1Qbv standard [2] prescribes Time Aware Shaper (TAS) scheduling algorithm to be implemented on the switches for obtaining a bounded latency for scheduled traffic. TSN switches implement Cycle Time (CT), where a few time slots in each CT are dedicated to the ST flows and the rest support the BE flows. The TAS ensures that a Gate Control Logic (GCL) implemented on egress ethernet ports provides the necessary end-to-end bounded latency for the ST flows. All TSN switches in the network are synchronized to nanosecond accuracy. As far as the reliability of ST is concerned, the network topology should offer mechanisms for Frame Replication and Elimination for Reliability (FRER). FRER involves duplicating data frames of ST flows at the source, routing them through different links, and filtering out the duplicates at the destination in line with IEEE 802.1CB [3]. Traditional application layer protocols that rely on TCP to provide reliability are not suitable for Tactile Internet traffic, as the inherent dependence of TCP on the retransmission of lost segments and timeouts increases latency. In particular, FRER is essentially a Layer-2 standard, and the upper layers of the network stack must be agnostic to its presence.

SmartNICs which function at Layer-2 offer to host several network processing functions traditionally performed by the CPU. In our implementation, we employ SmartNICs supporting P4 [4] and Micro-C which enable packet processing in the data plane. The algorithms presented in this paper can be deployed end-to-end or across a portion of the network. The novelty in this work lies in the fact that packet duplication and de-duplication are enabled without the involvement of the OS kernel. The implementation and demonstrations presented in this paper involve wired links, but the same can be extended to wireless links as well. Our contributions are as follows:

- We design and implement an intelligent packet de-duplication algorithm that works at wire speed.
- We develop a testbed to show end-to-end reliability.
- We demonstrate the working of FRER in presence of other flows in the network.

II. Literature Survey

Rayavarapu et al. [5] and Ergenç and Fischer [6] conducted extensive simulation studies to show that packet duplication may be initiated by considering a few MAC parameters from the user equipment (UE). These works do not present any test and measurement results. The focus in [6] and [7] is on the selection of disjoint paths on which FRER can be applied.

Packet de-duplication in real-time is a challenging problem in the implementation of FRER. There have been several attempts of implementation and demonstration of the effectiveness of FRER using simulation tools such as OMNet++ [7] and UUNET [8], but very few hardware implementations have

This work was supported jointly by the Ministry of Electronics and Information Technology, Government of India (SP/MITO-20-0006) and Centre for Networked Intelligence (a Cisco CSR initiative) at the Indian Institute of Science, Bengaluru.

been attempted till date. For a software implementation in the OS kernel of the end devices or switches, the expected processing time can be a few microseconds. But this number is excessive for core switches in a network which process multi-gigabits of traffic. The motivation for hardware implementation on a SmartNIC is to support wire-rate packet de-duplication with per packet processing time of the order of a few nanoseconds for end hosts as well as core switches/routers.

Ruiz et al. [9] present a de-duplication algorithm on FPGAs using 1024 sliding windows of dual port BRAMs, each containing 64-78 elements. 64-bit hash value of packet payload is employed; the first 10 bits are used to select the sliding window and the remaining 54 bits are used for lookup. The algorithm works in two clock cycles at 322 MHz. Stephan [10] presents an approach of performing packet de-duplication in FPGAs using P4. It involves a multicolumn hash table of size 65536. At the ingress, the hash values for the packets (also called signatures) are computed and looked up in the hash table; a match indicates a duplicate packet which is marked for drop. The table is updated by sliding the row (obtained by hash lookup) towards the right, popping out the rightmost column element, and populating the leftmost column element with the current packet signature. Their results with a four-column hash table indicate that 99% duplicates arriving within a delay of 40000 packets are filtered out. Unlike these de-duplication mechanisms which focus on building network monitoring tools for switches and routers to support port mirroring, our solution is targeted at improving the reliability of real-time flows. The implemented FRER mechanism works seamlessly both over a single link as well as end-to-end source and destination.

In an FPGA, it is possible to evaluate the hash of the entire payload and use it to identify duplicates. Also, single clock-cycle comparisons can be done with multiple elements as in [9]. But, for a target with pipelined switch architecture, e.g., Netronome Agilio SmartNICs, it is not possible to evaluate the hash of the entire payload due to memory constraints and limitations of supported P4 constructs. In that case, the algorithm has to rely on fields of the network headers such as IP Identification, TCP checksum etc to identify duplicate packets. Also, in addition to that, a comparison of a value with multiple keys in a single clock cycle is not possible. Considering these limitations, we propose a modified approach for such hardware targets. The algorithms presented in the following sections involve emulation of a hash table using an array which is used for the fast lookup of duplicates from a historic window of packet sequence numbers. This guarantees to filter out duplicates that arrive with delays lower than the window size. Further, we reduce the lookup time by using 16-bit sequence numbers in the packets as elaborated in Section III, and thus alleviate the requirement of performing computationally expensive hashing operations. Our algorithms can be implemented at the end hosts or at the core switches whereas the implementations in [9], [10] are targeted only towards the core switches.

III. Packet Duplication and Sequence Number Generation

We utilise the resources of the SmartNIC at the source to clone packets and forward them to alternate links. We develop P4 programs that exploit these capabilities, and thus significantly reduce the latency compared to the scenario where duplication is handled by the OS kernel.

Depending on the link quality and importance of the data, duplication can either be carried out conditionally or unconditionally. Conditional duplication involves real-time in-band network telemetry to estimate link quality [5], [12], based on which the SmartNIC decides to duplicate packets. This may be applied to video streaming applications where low frame drops might not significantly reduce the streaming experience. Such a strategy can be used to provide a tradeoff between link overloading and reliability.

For applications such as remote surgery where a teleoperator has to move a haptic device, and the trajectory of the device is required to be transmitted to the robot performing the surgery, it is expected that there be no packet loss and in this case, unconditional frame replication is an appropriate strategy.

Our implementation is based on IEEE 802.1CB standard [3] which specifies the usage of a sequence number in the range [0,65535] for the identification and elimination of duplicates. The initial sequence number at the start of communication is chosen uniformly at random from the set. Subsequent sequence numbers are obtained by incrementing the previous ones and rolling around once the maximum limit is reached. We envisage the following two cases:

1) In the case where FRER is deployed across two neighbouring nodes, our implementation uses the identification field of the IP header for sequence identification.
2) For end-to-end deployment, where packets typically undergo Network Address Translators (NATs) or Virtual Private Network(VPN) tunnel, the IP headers are modified. In this case, our implementation sets aside the first two bytes of the payload space for adding the sequence number.

Our experimental setup and the tests show performance across two hosts connected by two links. The de-duplication algorithms discussed in Section IV remain the same in the other case as well. An inherent assumption in our implementation is that none of the packets is fragmented.

IV. Packet De-Duplication

Since we use P4 programming language for de-duplication, we rely on P4 extern functions written in Micro-C to obtain support for loops. Arrays and hash tables are implemented using registers which are parts of P4 core library; the Micro-C routines use these registers to execute the algorithms listed below. All the duplicate packets are eliminated.

A. Sliding Window Based De-Duplication

The sliding-window approach is the simplest possible de-duplication algorithm. It uses an array of n registers (window) where the FRER sequence numbers of the last n (FRER sequence recovery history length [3]) packets are stored. A variable named *oldest_index* tracks the oldest FRER sequence number in the window. At each packet ingress, its FRER sequence number is compared with all the array contents. A match indicates a duplicate packet and no match indicates an original packet. In case of an original packet, its sequence

number substitutes *window[oldest_index]*, and *oldest_index* is incremented. The algorithm is implemented in Micro-C and used as an extern in the P4 code; the registers defined in P4 can be accessed in Micro-C as global variables. The pseudo-code for the algorithm is presented in Algorithm 1.

Algorithm 1 Sliding Window based De-Duplication

Input: window[n], n, oldest_index
Output: drop_packet / accept_packet

```
   /* set lookup key as FRER sequence no. */
 1: lookup_key = frer_sequence_no;
   /* loop through the window and drop packet
   if match found */
 2: for i = 0 to n − 1 do
 3:    if (window[i] == lookup_key) then
 4:       return drop_packet;
 5:    end if
 6: end for
   /* update window if no match found */
 7: window[oldest_index] = lookup_key;
   /* update index of oldest window element */
 8: oldest_index = (oldest_index+1)%n;
 9: return accept_packet;
```

An advantage of this algorithm is that it is deterministic for packet delays less than n and consumes less memory. It is suitable for handling intermittent stream traffic [3] where a small window size is required. Since this algorithm has a time complexity of $O(n)$, the packet processing latency grows linearly with the window size, making it difficult to scale up.

B. Hash Table Based De-Duplication

Large window size is desirable to get good de-duplication when alternate links have a significant difference in latencies. There may be multiple packets in the flight (e.g., in the case of bulk stream traffic) on both links, and the slower link may be carrying multiple stale packets which are duplicates.

The hash table based algorithm emulates a hash table using an array to enable fast window lookup while preserving the deterministic duplicate detection property. An array of size 65536 (initially set to 0) is used as a hash table, along with a sliding window of size n as in Section IV-A. At the ingress, the FRER sequence number of the packet is searched in the hash table. A value of 0 indicates that the packet is original and the window along with the hash table is updated. On the other hand, 1 indicates a duplicate packet which is dropped. The pseudocode for the algorithm is presented in Algorithm 2.

When performing de-duplication on a single stream, this algorithm can filter duplicates that arrive in order and within a delay of n with a hard guarantee. This approach has a $O(1)$ time complexity and uses $4(65536+n)$ bytes of register space, thereby trading off packet processing speed with larger NIC memory usage. The fixed window is required to prevent the overflow of the hash table. Since the sequence numbers repeat after the reception of 65536 packets, if only the hash table is used then all the original packets also get dropped. A finite window as in Section IV-A is used to avoid such a situation by refreshing the hash table states for older packets. The required window size for the deployment of this algorithm is a product of the supported bandwidth in packets/s and the difference in latencies of the two links.

Algorithm 2 Hash Table based De-Duplication

Input: window[n], hash_table[65536], n, oldest_index
Output: drop_packet / accept_packet

```
   /* set lookup key as FRER sequence no. */;
 1: lookup_key = frer_sequence_no;
   /* lookup in the hash table. if the lookup
   returns 1, then drop packet */
 2: if (hash_table[lookup_key] == 1 ) then
 3:    return drop_packet;
 4: end if
   /* add a new entry to hash table */
 5: hash_table[lookup_key] = 1;
   /* clear oldest entry from hash table */
 6: hash_table[window[oldest_index]] = 0;
   /* update window */
 7: window[oldest_index] = lookup_key;
   /* update index of oldest window element */
 8: oldest_index = (oldest_index+1)%n;
 9: return accept_packet;
```

C. Hash Collision Resolution with Stream Identification

The approach discussed in Section IV-B has a scalability issue due to potential hash collisions. For example, if de-duplication is to be performed on multiple disjoint streams, the sequence number chosen by one stream can coincide with another stream. Therefore, stream identification [3] (stream ID) is essential to avoid packet losses. One approach for stream identification could be to use IP layer fields. In our implementation, we use IEEE 802.1Q [14] VLAN IDs as stream identifiers assigned to the Ethernet frames. Our implementation, as shown in Algorithm 3, imposes a condition that the bitwise-AND of any two stream IDs be 0. As the VLAN ID is 12 bits in length, one instance of this algorithm can handle de-duplication for at most 12 streams with their VLAN IDs being $\{1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024, 2048\}$. A similar implementation using source IP address in a subnet with a netmask of 255.255.255.0 can support de-duplication for 8 hosts whose last octet of IP address are $\{1, 2, 4, 8, 16, 32, 64, 128\}$. Bitwise-AND of hash table entry with the VLAN ID is used to identify duplicate packets. Addition to the hash table is performed using bitwise OR with the VLAN ID. Similarly, hash table deletion is implemented using bitwise AND with the 1's complement of VLAN ID. The window has 2 columns - the first column stores the sequence number, and the second column stores the stream ID. This approach also has a $O(1)$ time complexity and uses $4(65536 + 2n)$ bytes of register space.

Table I compares the time complexities and register space usage of the above algorithms. Since the SmartNIC has a 4 byte word-aligned memory, each register uses 4 bytes of space.

Algorithm 3 Hash Table based De-Duplication with Collision Avoidance of Sequence Numbers

Input: window[n][2], hash_table[65536] ,n , oldest_index
Output: drop_packet / accept_packet

```
    /* set lookup key as FRER sequence no. */
1:  lookup_key = frer_sequence_no;
    /* set stream identifier as VLAN ID */
2:  stream_id = vlan_id;
    /* check if packet is received */
3:  if (hash_table[lookup_key] & stream_id == stream_id)
    then
4:      return drop_packet;
5:  end if
    /* clear oldest entry from hash table */
    /* | : bitwise OR, ! : 1's complement */
6:  hash_table[window[oldest_index][0]] &= !(stream_id);
    /* update window */
7:  window[oldest_index][0] = lookup_key;
8:  window[oldest_index][1] = stream_id;
    /* update index of oldest window element */
9:  oldest_index = (oldest_index+1)%n;
    /* update hash table with new stream id */
10: hash_table[lookup_key] |= stream_id;
11: return accept_packet;
```

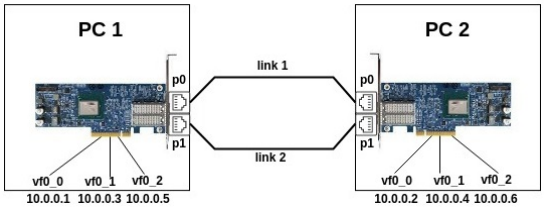

Fig. 1. Experimental Setup Network Topology

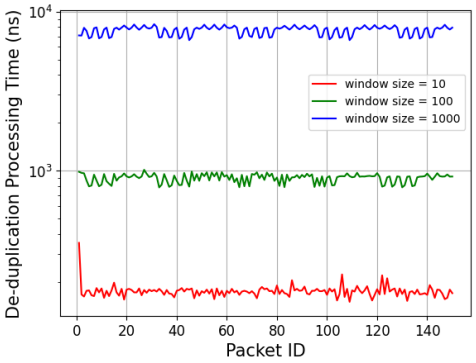

Fig. 2. De-duplication processing time for different window sizes for Sliding Window based approach

TABLE I
COMPARISON OF TIME AND SPACE COMPLEXITIES OF THE THREE APPROACHES

Algorithm	Parameters	Time Complexity	Register Space (Bytes)
Sliding window	window size = n	$O(n)$	$4n$
Hash table	window size = n	$O(1)$	$4(65536+n)$
Hash table with collision avoidance	window size = n	$O(1)$	$4(65536+2n)$

V. EXPERIMENTAL SETUP

Figure 1 shows the network topology used to test the algorithms discussed in Section IV. Two PCs, PC1 and PC2, each having Netronome Agilio 2x10GbE SmartNICs installed with virtual interfaces vf0_0, vf0_1 and vf0_2 and two physical interfaces p0 and p1 are connected via 10GbE fibre-optic cables as shown. PC1 is the transmitter that runs the P4 program to duplicate data packets ingressing from vf0_0 and routes the original packet through one link and their clones through the other. PC2 is the receiver running the P4 program that invokes a micro-C extern for de-duplication.

ST flows originating from vf0_0 are duplicated and forwarded to p0 and p1. Further, to analyse the performance of FRER in presence of BE flows, interfaces vf0_1 and vf0_2 are used to run BE flows through p0 and p1, respectively.

VI. LATENCY TEST RESULTS

For latency measurements, ScaPy library from Python is used to craft ST packets with two 64-bit metadata fields at the source (PC1), which is then duplicated and forwarded to the destination (PC2). The destination SmartNIC running the de-duplication algorithm updates the ingress and egress hardware timestamps in the metadata fields. The difference between the two provides the processing time consumed by the de-duplication algorithm. Figure 2 shows the de-duplication processing time for window sizes 10, 100, and 1000 using the sliding window algorithm. The de-duplication processing time increases linearly with the window size and has an average value of 175 ns for window size 10. Figures 3 and 4 show the de-duplication processing times for the hash table algorithms IV-B, IV-C respectively. The window sizes chosen are 100, 1000, and 10000. The average latency is comparable across the window sizes, with an average of 190 ns for single stream and 240 ns for multi-stream. Unlike the 6-ns latency (equivalent to 100 Gbps data rate) measured by [9], our single stream latency results show that the supported data rate for real-time flows is 2.52 Gbps (considering 60 byte packets).

A summary of the plots presented in Table II attests to the time complexity analysis presented in Table I. This affirms the feasibility of the hash table based de-duplication algorithm for large window sizes required for handling bulk streams. Furthermore, the sliding window approach is suitable for intermittent streams due to lesser memory usage while offering latencies similar to the hash table approach. Also, in general, while link redundancy can reduce the need for retransmissions, our algorithms at the SmartNIC offers low latency de-duplication and thus do not contribute significantly to retransmission timeouts.

VII. DE-DUPLICATION RESULTS FOR SINGLE STREAM

A. Packet Loss for ST flows in the absence of BE flows

Using FRER to achieve reliability relies on the assumption that at least one of the two (or more, depending upon the

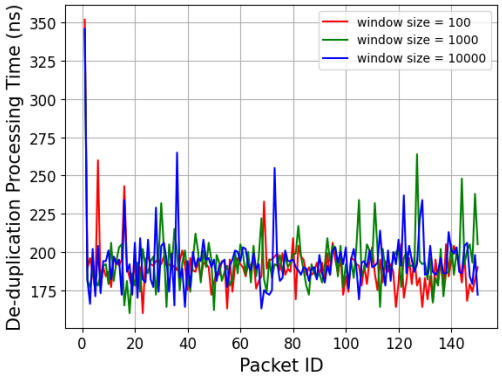

Fig. 3. De-duplication processing time for different window sizes for Hash Table based approach

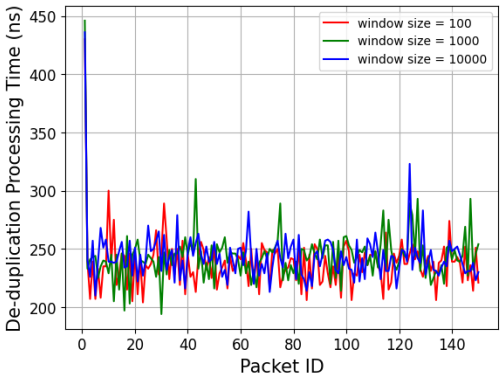

Fig. 4. De-duplication processing time for different window sizes for Hash Table based approach with Stream Identification

implementation) paths shall deliver the data packet to the end host. To illustrate the efficiency of FRER, packet losses were artificially introduced in the source SmartNIC (installed in PC1, Figure 1) with the help of Micro-C externs. The link connecting p0 of PC1 NIC to p0 of PC2 NIC was configured to drop one out of every 8 packets (12.5% loss), and the other link to drop one out of every 12 packets (8.3% loss) periodically depending on a counter value. Initial counter values of the individual links were chosen so as to ensure that no packet is dropped by both links.

The tests involved using Python socket client-server programs for sending 1,00,000 nos. of UDP datagrams of size 1.4Kbyte from vf0_0-PC1 at 11.2Mbits/s, all of which are duplicated and sent on the two links. Link 1 and Link 2 drop 12500 and 8333 packets respectively. The payload of each datagram was tagged with a sequence number to identify duplicates and missing packets. The test was run using the sliding window approach for window size 10 as well as the hash table based de-duplication algorithm with window sizes 100, 1000 and 10000. Each test was run 5 times, and the average results are presented in Table III. In certain cases, random packet losses were observed in the destination SmartNIC and reported in its drop counters (e.g., for hash table window size 10000, 2 packets are lost).

TABLE II
COMPARISON OF PROCESSING TIME VERSUS WINDOW SIZE

Algorithm	Processing Time (ns) (Window Size 100)	Processing Time (ns) (Window Size 1000)
Sliding Window	900 ± 58	7643 ± 484
Hash Table	190 ± 18	194 ± 20
Hash Table with stream identification	237 ± 23	244 ± 24

TABLE III
RELIABILITY TEST RESULTS WITH PERIODIC PACKET LOSSES

	Sliding Window (Window Size=10)	Hash Table (Window Size = 100)	Hash Table (Window Size = 1000)	Hash Table (Window Size = 10000)
Original packets received	100000	100000	100000	99998
Duplicate packets received	0	0	0	0

B. *No Packet Loss in the presence of other traffic*

Depending on deployment, end hosts as well as relay systems need to process the de-duplication algorithms in presence of other flows. This is illustrated in the following paragraph.

For this test, 100Mb/s TCP BE flows ran between vf0_1-PC1 — vf0_1-PC2 and vf0_2-PC1 — vf0_2-PC2 via p0 and p1, respectively. Similar to Section VII-A, $1,00,000$ nos. of 1.4KB datagrams were sent from vf0_0-PC1 to vf0_0-PC2 at 11.2Mb/s and the sent and received sequence number logs are analysed. The packet loss scenario is not emulated. This test is to demonstrate the efficacy of the de-duplication algorithm under the highest possible load (all duplicates reach the destination) in the presence of multiple flows. For each window size, the numbers of received original and duplicate packets remain the same as in the case of single flow (see Table III). The SmartNIC has a 40Gb/s packet processing capacity; as long as the combined bandwidth utilised by the flows is lower than this capacity, the similarity in results is expected.

VIII. DE-DUPLICATION RESULTS FOR MULTIPLE STREAMS

To test the effectiveness of our implementation of multi-stream deduplication in Algorithm 3, a PCAP file of 786432 packets i,e., 65,536 packets each for 12 VLAN IDs was generated and sent using the tcpreplay tool. Packets with the same sequence number were transmitted in a contiguous block. Using our user-defined P4 registers, statistics of number of original packets dropped and duplicates received were obtained. Ideally, all duplicates that arrive within a delay lesser than the window size (FRER sequence recovery history length) should be filtered out. However, the test results in Figures 5 and 6 do show that some original packets ($\leq 0.2\%$) are lost and some duplicates ($\leq 0.2\%$) are accepted. The results are summarised in Table IV in terms of duplicate detection efficiency and packet delivery ratio. For a 1 Gigabit flow, the obtained de-duplication efficiency is 99.8777% and the packet delivery ratio is 99.8347%.

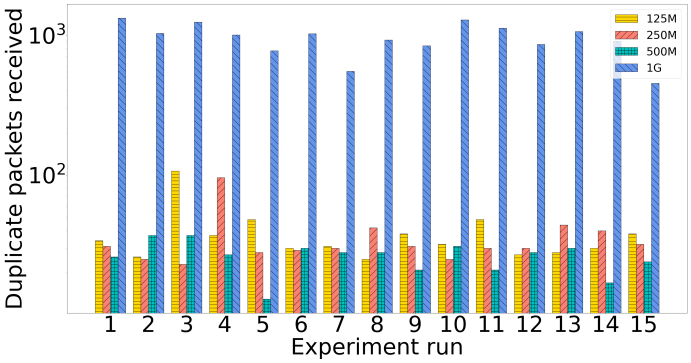

Fig. 5. Number of Duplicate Packets missed per run of test versus bandwidth

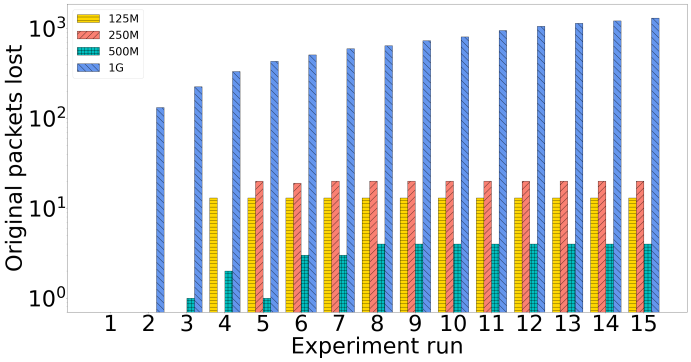

Fig. 6. Number of Original Packets lost per run of test versus bandwidth

For a lower packet size (400 Bytes), higher packet losses are observed. This is because the SmartNIC stores packet headers and payload in separate memory sections until the packet is deparsed and emitted. For the same bandwidth, lower packet size translates to higher demand on packet processing rate which leads to higher packet losses. From Figure 6, it can be observed that at reset the packet losses are minimum, then they increase monotonically before settling at a steady value. Implementation of FRER sequence recovery history timeout [3] for flushing out the window and hash table entries at periodic intervals will help in reducing the average packet loss.

IX. Conclusion and Future Work

The hardware implementation on the SmartNIC, along with the results presented in VII, is a clear qualitative testimony demonstrating how FRER helps in enhancing the reliability between two hosts in a TSN network. Even though the presented implementation involves wired links, the implementation can be extended to a wireless scenario as well. When coupled with other TSN standards like IEEE 802.1AS [15] and IEEE 802.1Qbv [2], FRER ensures high-reliability and low latency communication for tactile internet applications.

Since network conditions can vary over a period of time, we require mechanisms to dynamically size the window so that there is efficient usage of memory. Unfortunately, the current hardware supports this over a firmware reload. A smaller window size per flow can also allow the hash table de-duplication algorithm to process multiple flows with lesser overhead for hash-collision. Also, attempts to integrate multiple TSN protocols in a single hardware target to build a full-fledged TSN switch shall be a giant leap ahead towards the future of uRLLC, 5G and beyond.

TABLE IV
Performance Summary of Multistream De-Duplication for 1400 Byte Packets

Bandwidth (Mbits/s)	Duplicate Detection Efficiency	Packet Delivery Ratio
125	99.9951	99.9983
250	99.9955	99.9975
500	99.9966	99.9995
1000	99.8777	99.8347

References

[1] 3GPP TS 22.104 V17.7.0, 3rd Generation Partnership Project, September 2021

[2] "IEEE Standard for Local and metropolitan area networks – Bridges and Bridged Networks - Amendment 25: Enhancements for Scheduled Traffic," in IEEE Std 802.1Qbv-2015 (Amendment to IEEE Std 802.1Q-2014 as amended by IEEE Std 802.1Qca-2015, IEEE Std 802.1Qcd-2015, and IEEE Std 802.1Q-2014/Cor 1-2015), vol., no., pp.1-57, 18 March 2016, doi: 10.1109/IEEESTD.2016.8613095.

[3] "IEEE Standard for Local and metropolitan area networks–Frame Replication and Elimination for Reliability," in IEEE Std 802.1CB-2017, vol., no., pp.1-102, 27 Oct. 2017, doi: 10.1109/IEEESTD.2017.8091139.

[4] P4 Language Consortium, tutorials on P4, https://github.com/p4lang/tutorials

[5] S. M. Rayavarapu, S. D. Amuru and K. Kiran, "Dynamic Control of Packet Duplication in 5G-NR Dual Connectivity Architecture," 2020 International Conference on COMmunication Systems & NETworkS (COMSNETS), Bengaluru, India, 2020, pp. 539-542, doi: 10.1109/COMSNETS48256.2020.9027289.

[6] D. Ergenç and M. Fischer, "Implementation and Orchestration of IEEE 802.1CB FRER in OMNeT++," 2021 IEEE International Conference on Communications Workshops (ICC Workshops), Montreal, QC, Canada, 2021, pp. 1-6, doi: 10.1109/ICCWorkshops50388.2021.9473722.

[7] D. Ergenç and M. Fischer, "On the Reliability of IEEE 802.1CB FRER," IEEE INFOCOM 2021 - IEEE Conference on Computer Communications, Vancouver, BC, Canada, 2021, pp. 1-10, doi: 10.1109/INFOCOM42981.2021.9488750.

[8] T. Tan and S. -k. Park, "Redundancy path implementation for schedule traffic," 2018 International Workshop on Advanced Image Technology (IWAIT), Chiang Mai, Thailand, 2018, pp. 1-3, doi: 10.1109/IWAIT.2018.8369737.

[9] M. Ruiz, G. Sutter, S. López-Buedo, J. F. Zazo and J. E. López de Vergara, "An FPGA-based approach for packet deduplication in 100 gigabit-per-second networks," 2017 International Conference on ReConFigurable Computing and FPGAs (ReConFig), Cancun, Mexico, 2017, pp. 1-6, doi: 10.1109/RECONFIG.2017.8279776.

[10] Stefan Johannson, "Packet Deduplication in P4", Open Networking Foundation, P4 Workshop May 2021, https://opennetworking.org/wp-content/uploads/2021/05/2021-P4-WS-Stefan-Johansson-Slides.pdf.

[11] J. Fang, S. Sudhakaran, D. Cavalcanti, C. Cordeiro and C. Chen, "Wireless TSN with Multi-Radio Wi-Fi," 2021 IEEE Conference on Standards for Communications and Networking (CSCN), Thessaloniki, Greece, 2021, pp. 105-110, doi: 10.1109/CSCN53733.2021.9686180.

[12] Lianglu Yin, Yuanliang Huang, Weixiong Chen, "Reliability analysis of wireless communication based on ZigBee", ICEEECS 2016, https://doi.org/10.2991/iceeecs-16.2016.14.

[13] Cormen, Thomas H.; Leiserson, Charles E.; Rivest, Ronald L.; Stein, Clifford (2009). Introduction to Algorithms (3rd ed.). Massachusetts Institute of Technology. pp. 253–280. ISBN 978-0-262-03384-8.

[14] "IEEE Standard for Local and metropolitan area networks–Bridges and Bridged Networks," in IEEE Std 802.1Q-2014 (Revision of IEEE Std 802.1Q-2011) , vol., no., pp.1-1832, 19 Dec. 2014, doi: 10.1109/IEEESTD.2014.6991462.

[15] "IEEE Standard for Local and Metropolitan Area Networks–Timing and Synchronization for Time-Sensitive Applications," in IEEE Std 802.1AS-2020 (Revision of IEEE Std 802.1AS-2011) , vol., no., pp.1-421, 19 June 2020, doi: 10.1109/IEEESTD.2020.9121845.